České vysoké učení technické v Praze
Fakulta jaderná a fyzikálně inženýrská

Název práce česky název práce česky název práce česky název práce česky

Title of the Work in English Title of the Work in English Title of the Work in English

Bakalářská práce

Autor: **Jméno Autora**

Vedoucí práce: **prof. Ing. Jméno Školitele, DrSc.**

Konzultant: **doc. RNDr. Jméno Konzultanta, CSc.** (pouze pokud konzultant byl jmenován.)

Akademický rok: 2024/2025

ZADÁNÍ BAKALÁŘSKÉ PRÁCE

I. OSOBNÍ A STUDIJNÍ ÚDAJE

Příjmení: Příjmení Jméno: **Jméno** Osobní číslo:

Fakulta/ústav: **Fakulta jaderná a fyzikálně inženýrská**

Zadávající katedra/ústav: **Katedra matematiky**

Studijní program: **(není nutné vyplňovat, doplní se po přepsání do systému)**

Specializace: **(není nutné vyplňovat, doplní se po přepsání do systému)**

II. ÚDAJE K BAKALÁŘSKÉ PRÁCI

Název bakalářské práce:

Šablona

Název bakalářské práce anglicky:

Template

Pokyny pro vypracování:

1. první bod zadání
2. druhý bod zadání
3. atd.

Místo tohoto souboru použijte soubor PDF se všemi podpisy, který lze stáhnout z KOS, resp. jste jej obdrželi e-mailem.

Seznam doporučené literatury:

[1] reference č. 1
[2] reference č. 2
[3] atd.

Jméno a pracoviště vedoucí(ho) bakalářské práce:

prof. Ing. Jméno Školitele, DrSc.

Jméno a pracoviště druhé(ho) vedoucí(ho) nebo konzultanta(ky) bakalářské práce:

doc. RNDr. Jméno Konzultanta, CSc.

Datum zadání bakalářské práce: **31.10.2023** Termín odevzdání bakalářské práce: **05.08.2024**

Platnost zadání bakalářské práce: **30.09.2025**

(jméno vedoucího - bude doplněno) podpis vedoucí(ho) práce	(jméno vedoucího - bude doplněno) podpis vedoucí(ho) ústavu/katedry	doc. Ing. Václav Čuba, Ph.D. podpis děkana

© ČVUT v Praze, Design: ČVUT v Praze, VIC

III. PŘEVZETÍ ZADÁNÍ

Student bere na vědomí, že je povinen vypracovat bakalářskou práci samostatně, bez cizí pomoci, s výjimkou poskytnutých konzultací.
Seznam použité literatury, jiných pramenů a jmen konzultantů je třeba uvést v bakalářské práci.

.	
Datum převzetí zadání	Podpis studenta

Místo tohoto souboru použijte soubor PDF se všemi podpisy, který lze stáhnout z KOS, resp. jste jej obdrželi e-mailem.

© ČVUT v Praze, Design: ČVUT v Praze, VIC

FAKULTA JADERNÁ A FYZIKÁLNĚ INŽENÝRSKÁ
FACULTY OF NUCLEAR SCIENCES AND PHYSICAL ENGINEERING
Břehová 7
115 19 Praha 1

PROHLÁŠENÍ

Já, níže podepsaný

Příjmení, jméno studenta:	John Doe
Osobní číslo:	123456
Název programu:	Matematické inženýrství

prohlašuji, že jsem bakalářskou práci s názvem

... název práce ...

vypracoval samostatně a uvedl veškeré použité informační zdroje v souladu s Metodickým pokynem o dodržování etických principů při přípravě vysokoškolských závěrečných prací a Rámcovými pravidly používání umělé inteligence na ČVUT pro studijní a pedagogické účely v Bc a NM studiu.

Prohlašuji, že jsem v průběhu příprav a psaní závěrečné práce použil nástroje umělé inteligence. Vygenerovaný obsah jsem ověřil. Stvrzuji, že jsem si vědom, že za obsah závěrečné práce plně zodpovídám.

V Praze dne .....

John Doe
.....
podpis studenta

Místo tohoto souboru použijte prohlášení vygenerované a podepsané v systému KOS.

Poděkování:
Chtěl bych zde poděkovat především svému školiteli ..... za pečlivost, ochotu, vstřícnost a odborné i lidské zázemí při vedení mé diplomové práce. Dále děkuji svému konzultantovi ..... za .....

Název práce:
Název práce

Autor: Jméno Autora

Studijní program: Celý název studijního programu (nikoliv zkratka)

Specializace: Celý název specializace (Pokud se studijní program nedělí na specializace, tuto řádku odstranit.)

Druh práce: Bakalářská práce

Vedoucí práce: prof. Ing. Jméno Školitele, DrSc., pracoviště školitele (název instituce, fakulty, katedry...)

Konzultant: doc. RNDr. Jméno Konzultanta, CSc., pracoviště konzultanta. Pouze pokud konzultant byl jmenován.

Abstrakt: Abstrakt max. na 10 řádků.

Klíčová slova: klíčová slova (nebo výrazy) seřazená podle abecedy a oddělená čárkou

Title:
Title of the Work

Author: Author's Name

Abstract: Max. 10 lines of English abstract text.

Key words: keywords in alphabetical order separated by commas

Obsah

Úvod **12**

1 Bayseovská statistika **13**
1.1 Histrorie Bayesovské statistiky 13
1.2 Základní principy Bayesovské statistiky 13

2 Problematika výběru modelů **14**
2.1 Obecný princip hledání modelu 14
2.2 Baysevův přístup k výběru modelu 15
2.3 GP regrese 16
2.4 Výběr modelu pro GP regresi 18
2.4.1 Type II ML 18
2.4.2 Markov Chain Monte Carlo 19

3 Prior–Data Fitted Networks **21**
3.1 Úvod do Prior–Data Fitted Networks 21
3.2 Vnitřní architektura PFN 22
3.2.1 Fáze 1 - Meta–Learning 23
3.2.2 Fáze 2 - Vstup do modelu 24
3.2.3 Fáze 3 – Transformer vrstvy 26
3.2.4 Fáze 4 – Dekóder, převod embdedingu na logity. 28
3.3 Teoretické statistické vlastnosti PFN 30

Závěr **31**

Úvod

vns. fvffdbeb bdsbf
Text úvodu....

Kapitola 1

Bayseovská statistika

1.1 Histrorie Bayesovské statistiky

1.2 Základní principy Bayesovské statistiky

Kapitola 2

Problematika výběru modelů

V této kapitole uvedeme čtenáře do problematiky výběru modelu. Ukážeme si zde jaké různé přístupy výběru modelu při modelování dat existují, jaké potíže se s tím vážou a lze-li tyto potíže elegantně vyřešit. V praxi se často setkáváme se situacemi, kdy musíme pro daná data zvolit nejvhodnější model z několika kandidátů. Například při analýze časových řad můžeme mít několik různých modelů, které by mohly popsat pozorovaná data, a potřebujeme rozhodnout, který z konkrétní model z funkčního prostoru je nejvhodnější. Dnes existuje celá řada metod, jak tento konkrétní model určit, jak najít vhodné parametry modelu, ovšem všechny tyto metody jsou zpravidla dost výpočetně náročné. Avšak se to změnilo s příchodem prudkého rozvoje neuronových sítí a hlubokého učení. O Neuronových sítích víme, že to je dobrý aproximační nástroj a ukázalo se totiž, že abychom mohli co nejefektivněji aproximovat hledaný model pro naše data, lze použít architekturu Transformer, která, jak se postupem času dozvíme, je nejpřesnějším a nejrychlejším řešením tohoto problému. Zavedeme nejdříve obecnou problematiku výběru modelu, poté Bayesovský princip výběru modelu a nakonec si ukážeme, jak nám v tom může pomoci architektura Transformer.

2.1 Obecný princip hledání modelu

V této sekci si ukážeme obecný princip výběru modelu. Představme si následující situaci: máme k dispozici naměřená data, např. z fyzikálního experimentu nebo ze sociální studie a chceme najít model, který tato data co nejlépe popisuje. Tento model může být například lineární nebo nelineární regrese, rozhodovací strom nebo neuronová síť. Dalším účelem modelu je schopnost generalizace na nová, neviděná data, tj. možnost predikce nebo možnost otestovat pravdivost modelu. V tomto okamžiku před námi stojí několik otázek:

- Jakou strukturu modelu musíme zvolit (např. lineární vs. polynomiální)
- Jak správně zvolit hodnoty parametrů a hyperparametrů modelu (např. síla regularizace nebo počet vrstev v neuronové síti)
- Jak moc složitý model zvolit, aby dobře fitoval trénovací data, ale zároveň dobře generalizoval na nová data

Nakonec z toho vyplývá i formulace daného problému: proces výběru modelu je proces systematického výběru optimálního modelu. Během tohoto procesu se vzniká i základní trade-off dilema. Zvolíme-li příliš jednoduchý model, nebude schopen zachytit strukturu dat a bude mít špatný fit i špatnou generalizaci (underfitting). Naopak zvolíme-li příliš složitý model, bude perfektně fitovat (interpolovat) trénovací

data, ale bude špatně generalizovat na nová data (overfitting). Cílem je najít optimální model z těchto hledisek.

Dnes existuje celá řada přístupů k výběru modelu. Jsou to tři hlavní přístupy: Bayesovský přístup (marginal likelihood), cross-validace a informační kritéria (AIC, BIC). Cross-validace je praktický a intuitivní přístup, který odhaduje generalizační chybu pomocí validačních dat. Jinými slovy, data rozdělíme na několik částí, pro každou část trénujeme model na ostatních datech a testujeme na ní. Nakonec průměrujeme chyby přes všechny části a vybereme model s tzv. nejnižší cross-validační chybou [Rasmussen]. Dalším velmi populárním přístupem je Bayesovský přístup, který spočívá v tom, že spočítáme pravděpodobnost modelu daná daty. To znamená, že spočítáme tzv. marginal likelihood, což je pravděpodobnost dat daná modelem, integrovaná přes všechny možné hodnoty parametrů modelu. Tento přístup má výhodu v tom, že automaticky penalizuje složité modely a nabízí principiální způsob, jak balancovat fit a složitost modelu. Posledním přístupem jsou informační kritéria, jako je Akaike Information Criterion (AIC) a Bayesian Information Criterion (BIC). Tyto metody poskytují rychlou a jednoduchou aproximaci marginal likelihood, která penalizuje složitost modelu. Nás avšak bude v této práci nejvíce zajímat Bayesovský přístup, protože nám poskytuje nejvíce principů pro výběr modelu a je základem pro náš další postup.

2.2 Baysevův přístup k výběru modelu

Zde ukážeme Baysevův princip při výběru modelu. Je zvykem použít hierarchický princip při zavádění Bayesovského výběru modelu [Rasmussen].

Označme tedy $\mathbf{w} \in \mathcal{W}$ jako parametry modelu, kde $\mathcal{W}$ je prostor parametrů modelu, mohou to být např. váhy v lineární regresi nebo v neuronové síti. Dále nechť $\boldsymbol{\theta} \in \Theta$ jsou hyperparametry modelu, kde Θ je prostor hyperparametrů modelu, např. počet vrstev v neuronové síti. Nakonec označme $\mathcal{H}_i \in \mathcal{H}$ jako různé struktury modelu, kde $\mathcal{H}$ je prostor modelů, např. různé architektury neuronových sítí. V Bayesovském přístupu k výběru modelu používáme tzv. Bayesovskou inferenci k odhadu pravděpodobnostního rozdělení parametrů, hyperparametrů a modelů na základě pozorovaných dat $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$, kde $\mathbf{x}_i$ jsou vstupní data a y_i jsou odpovídající cílové hodnoty, anebo alternativně lze označit jako $\mathcal{D} = \{(\mathbf{X}, \mathbf{y})\}$. Tj. cílem je na základě pozorovaných dat dojít k závěru, který model s kterými parametry a hyperparametry nejlépe vysvětluje pozorovaná data.

Na první úrovni hierarchie odhadneme pravděpodobnostní rozdělení parametrů modelu $\mathbf{w}$ pro dané (fixní) hyperparametry $\boldsymbol{\theta}$ a model $\mathcal{H}_i$. Zde používáme pojem posteriorního rozdělení, tj. ptáme se, jaké je pravděpodobnostní rozdělení parametrů modelu $\mathbf{w}$ vzhledem k pozorovaným datům $\mathcal{X}$, fixním hyperparametrům $\boldsymbol{\theta}$ a modelu $\mathcal{H}_i$. Předpokládejme, že chceme získat predikci pro vstupní data $\mathbf{y}$. Potom podle Bayesovy věty platí:

$$P(\mathbf{w}|\mathbf{y}, \mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i) = \frac{P(\mathbf{y}|\mathbf{w}, \mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i)P(\mathbf{w}|\boldsymbol{\theta}, \mathcal{H}_i)}{P(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i)}, \tag{2.1}$$

kde $P(\mathbf{y}|\mathbf{w}, \mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i)$ je pravděpodobnost dat vzhledem k parametrům modelu $\mathbf{w}$, vstupním datům $\mathbf{y}$, hyperparametrům $\boldsymbol{\theta}$ a modelu $\mathcal{H}_i$ (tzv. likelihood), $P(\mathbf{w}|\boldsymbol{\theta}, \mathcal{H}_i)$ je apriorní rozdělení parametrů modelu $\mathbf{w}$ vzhledem k hyperparametrům $\boldsymbol{\theta}$ a modelu $\mathcal{H}_i$, a $P(\mathbf{X}|\mathbf{y}, \boldsymbol{\theta}, \mathcal{H}_i)$ je tzv. evidence dat.

Na druhé úrovni hierarchie odhadneme pravděpodobnostní rozdělení hyperparametrů modelu $\boldsymbol{\theta}$ pro daný (fixní) model $\mathcal{H}_i$. Podobně jako na první úrovni používáme Bayesovu větu:

$$P(\boldsymbol{\theta}|\mathbf{y}, \mathbf{X}, \mathcal{H}_i) = \frac{P(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i)P(\boldsymbol{\theta}|\mathcal{H}_i)}{P(\mathbf{y}|\mathbf{X}, \mathcal{H}_i)}, \tag{2.2}$$

kde $P(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i)$ je pravděpodobnost dat vzhledem k vstupním datům $\mathbf{y}$, hyperparametrům $\boldsymbol{\theta}$ a modelu $\mathcal{H}_i$ (tzv. likelihood), $P(\boldsymbol{\theta}|\mathcal{H}_i)$ je apriorní rozdělení hyperparametrů modelu $\boldsymbol{\theta}$ vzhledem k modelu $\mathcal{H}_i$, a $P(\mathbf{y}|\mathbf{X}, \mathcal{H}_i)$ je evidence dat.

Na třetí úrovni hierarchie odhadneme pravděpodobnostní rozdělení modelů $\mathcal{H}_i$. Opět používáme Bayesovu větu:

$$P(\mathcal{H}_i|\mathbf{y}, \mathbf{X}) = \frac{P(\mathbf{y}|\mathbf{X}, \mathcal{H}_i)P(\mathcal{H}_i)}{P(\mathbf{y}|\mathbf{X})}, \tag{2.3}$$

kde $P(\mathbf{y}|\mathbf{X}, \mathcal{H}_i)$ je pravděpodobnost dat vzhledem k vstupním datům $\mathbf{y}$ a modelu $\mathcal{H}_i$ (tzv. likelihood), $P(\mathcal{H}_i)$ je apriorní rozdělení modelů $\mathcal{H}_i$, a $P(\mathbf{y}|\mathbf{X})$ je evidence dat.

Celý tento postup nám umožňuje systematicky vyhodnotit různé modely, jejich hyperparametry a parametry na základě pozorovaných dat a vybrat ten, který nejlépe vysvětluje data podle Bayesovského principu. Zde je dokonce vidět, proč jsme použili hierarchický přístup - na každé úrovni hierarchie odhadujeme pravděpodobnostní rozdělení na základě předchozí úrovně, což nám umožňuje efektivně zachytit nejistotu na různých úrovních modelování. Postupujeme tedy od nejvyšší úrovně nejistoty, což je vůbec výběr modelu, přes střední úroveň, což je výběr hyperparametrů, až k nejnižší úrovni, což je odhad parametrů modelu, tj. se náš problém na každé úrovni zúžuje, tedy jdeme do hloubky nastavení modelu.

Avšak v praxi při výpočtech hustot pravděpodobností na jednotlivých úrovních hierarchie můžeme narazit na problém, že ten integrál neumíme spočítat analyticky, např. protože tato hustota není normalizovaná (neboli tzv. nevlastní). Takovým integrálem je např. normalizační konstanta na první úrovni hierarchie:

$$P(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i) = \int_{\mathcal{W}} P(\mathbf{y}|\mathbf{w}, \mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i)P(\mathbf{w}|\boldsymbol{\theta}, \mathcal{H}_i)d\mathbf{w}, \tag{2.4}$$

anebo na druhé úrovni hierarchie:

$$P(\mathbf{y}|\mathbf{X}, \mathcal{H}_i) = \int_{\Theta} P(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i)P(\boldsymbol{\theta}|\mathcal{H}_i)d\boldsymbol{\theta} \tag{2.5}$$

Tyto integrály je nutné aproximovat numericky pomocí takových metod jako Markov chain Monte Carlo (MCMC) nebo pokročilejším No-U-Turn Sampler (NUTS), pokud chceme odhadnou integrál z nenormalizovaných hustot pravděpodobnostní, jako je integrál (2.5), nebo např. Type II maximum likelihood (Type II ML), chceme-li v odhadnout pouze posteriorní pravděpodobnost pro hyperparametry modelu, což spočívá v Laplaceově aproximaci integrálu (2.4) a následné maximalizaci $P(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i)$. Všechny tyto metody jsou však dost výpočetně náročné a může trvat značné množství času, než se podaří najít vhodný model pro daná data. Jedním z účelů této práce je ukázat, jak lze tyto pravděpodobnostní hustoty efektivně aproximovat pomocí úplně nového postupu, a to pomocí architektury Transformer.

2.3 GP regrese

Zde si ukážeme, jak aplikovat Bayesovský přístup výběru modelu na konkrétním příkladě, což bude prokládání dat pro nelineární regresi pomocí Gaussovských procesu (dále jen GP). Dále si ukážeme i základní metody pro hledání vhodného modelu, o kterých jsme zmínili v předchozí sekci, a to jsou Type II ML a MCMC. Avšak předtím zadefinujeme základní pojmy GP regresi.

Představme si tedy, že máme před sebou následující problém. Někdo nám donesl sadu dat, mohou to být biomedicínská data, data z ekonomických trhu nebo dokonce i data z fyzikálního experimentu a chce po nás sestavit model, který by popisoval tato data a pomocí kterého bychom mohli také stanovit závěry nebo predikce pro budoucí výsledky na základě aktuálních. Je zcela přirozené, že chceme najít co nejpřesnější model, který by tato data popisoval. Jedním z přístupů, jak toho dosáhnout, je použití Gaussovských procesů (GP) pro regresi. Nejdříve ve zkratce zavedeme pojem GP regresi.

Zvykem je definovat Gaussovský proces pomocí vícerozměrného normálního rozdělení. Mějme tedy vícerozměronou náhodnou veličinu $\mathbf{X} = (X_1, \ldots, X_n)$ a nechť $X_i \sim \mathcal{N}(\boldsymbol{\mu_i}, \sigma_i)$ pro $\forall i = 1 \ldots n$, pak říkáme, že $\mathbf{X}$ má vícerozměrné normální rozdělení s vektorem středních hodnot $\boldsymbol{\mu} = (\mu_1, \ldots, \mu_n)$ a kovarianční maticí Σ, pokud její hustota pravděpodobnosti je dána vztahem:

$$p(\mathbf{x}) = \frac{1}{(2\pi)^{n/2}|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^T \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu})\right), \tag{2.6}$$

kde $|\Sigma|$ je determinant matice Σ vektor $\mathbf{x}$ jsou vektor vstupních hodnot, značíme $\mathbf{X} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)$.

Z toho vztahu plyne i definice Gaussovského procesu [Rasmussen]:

Definice 2.1 (Gaussovský proces)**.** Gaussovský proces je soubor náhodných veličin, libovolný konečný počet kterých má sdružené normální rozdělení.

Dalším klíčovým pojmem je tzv. kovarianční funkce (nebo jádro), která určuje vlastnosti GP. Ta se definuje jako funkce $k : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$, která pro každé dva vstupy $\mathbf{x}, \mathbf{x}' \in \mathbb{R}^d$ vrací hodnotu $k(\mathbf{x}, \mathbf{x}')$, která udává míru podobnosti mezi těmito dvěma vstupy. Je to jistá modifikace kovarianční matice, která je částí klasického vícerozměrného normálního rozdělení, a které vznikne po aplikaci tzv. Kernel triku. Máme-li nějaký proces $f(\mathbf{x})$, teď stejný proces chceme zkoumat i pro jiný vstup, tj. nás zajímá $f(\mathbf{x}')$. Teď je před námi otázka, jaká je kovariance mezi $f(\mathbf{x})$ a $f(\mathbf{x}')$? Potom se skrz funkční prostor vah a kovarianční matice Σ ukáže, že kovarianční funkce je definovaná jako:

$$k(\mathbf{x}, \mathbf{x}') = \phi(\mathbf{x})^T \Sigma \phi(\mathbf{x}'), \tag{2.7}$$

kde $\phi(\mathbf{x})^T \Sigma \phi(\mathbf{x}')$ je součin v prostoru vah s kovarianční matici dvou různých vstupu.

Nakonec platí, že Gaussovský process je plně definován svou střední funkcí $m(\mathbf{x})$ a kovarianční funkcí $k(\mathbf{x}, \mathbf{x}')$. Střední funkci a kovarianční funkci tohoto procesu definujeme jako:

$$m(\mathbf{x}) = \mathbb{E}[f(\mathbf{x})], k(\mathbf{x}, \mathbf{x}') = \mathbf{E}[(f(\mathbf{x}) - m(\mathbf{x}))(f(\mathbf{x}) - m(\mathbf{x}))], \tag{2.8}$$

a následně říkáme, že proces $f(x)$ je Gaussovský proces a značíme $f(\mathbf{x}) \sim \mathcal{GP}(\boldsymbol{m}(\boldsymbol{(x)}), k((x), (x')))$.

Platí dále, že kovarianční funkce, které se také říká kernel (jádro), může mít různé formy. Mezi nejčastěji používané patří např. Radial Basis Function (RBF) kernel (někdy squared exponential kernel), který je definován jako:

$$k(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2l^2}\right), \tag{2.9}$$

kde σ_f^2 je rozptyl funkce a l je délka škály. Pak existuje spousta dalších kernelů, jako je Matérn kernel, Periodic kernel nebo Linear kernel, každý s různými vlastnostmi a aplikacemi, ale pro naše účely postačí základní RBF kernel. RBF kernel má řadu výhod: je hladký, nekonečně diferencovatelný a má tendenci dobře aproximovat širokou škálu funkcí. Současně platí, že každý datový bod můžeme představit jako střed Gaussovského rozdělení. Z předchozí myšlenky plyne i velmi intuitivní myšlenka o tom, jak funguje GP regrese: každý datový bod přispívá k predikci nového bodu podle své vzdálenosti od něj, přičemž váha tohoto přispění je určena kovarianční funkcí (jádrem). Tím pádem, čím blíž je nějaký bod k jinému bodu, tím větší vliv na něj má: body blízko sebe mají hodnotu blízkou 1, Vzdálené body mají hodnotu blízkou 0 a funguje to jako "měřítko podobnosti"mezi body. Pokud bychom tedy chtěli predikovat např. počasí, tak je jasno, že pokud před 5 hodinami bylo −6 stupňů Celsia, před dvěma hodinami −4 stupňů Celsia a před hodinou −5 stupně Celsia, tak je zjevné, že tyto hodnoty budou mít vliv na predikci počasí za hodinu a ta teplota bude někde mezi −6 a −4 stupňů Celsia a určitě nestoupne nebo neklesne o víc než 5 hodnot.

2.4 Výběr modelu pro GP regresi

V předchozí kapitole jsme si avšak ukázali, že hledat matematické modely, které přesně popisují data, není vůbec jednoduché nebo dokonce nemožné a je potřeba použít různé metody při hledání vhodného modelu. Hlavní potíží je výpočet integrálů, které se objevovali při sestavování Bayesovské hierarchie. Zde si ukážeme dvě základná a nejpopulárnější metody, které nám tyto výpočetní přehrady pomáhají obejít, anebo vyřešit.

2.4.1 Type II ML

Jednou z takových metod je Type II maximum likelihood (Type II ML). Tato metoda spočívá v tom, že maximalizujeme tzv. marginal likelihood, což je pravděpodobnost dat daná modelem, integrovaná přes všechny možné hodnoty parametrů modelu. V případě GP regrese je marginal likelihood dána vztahem:

$$P(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta}, \mathcal{H}) = \int_{\mathcal{W}} P(\mathbf{y}|\mathbf{w}, \mathbf{X}, \boldsymbol{\theta}, \mathcal{H}) P(\mathbf{w}|\boldsymbol{\theta}, \mathcal{H}) d\mathbf{w}, \tag{2.10}$$

kde $\boldsymbol{\theta}$ jsou hyperparametry modelu, $\mathcal{H}$ je struktura modelu, $\mathbf{w}$ jsou parametry modelu, $\mathbf{X}$ jsou vstupní data a $\mathbf{y}$ jsou odpovídající cílové hodnoty. To znamená následující, vzpomeňme si na 2. úroveň hierarchie v Bayesovském přístupu, tj. na rovnici 2.2. Pak posteriorní rozdělení odhadujeme takto:

$$P(\boldsymbol{\theta}|\mathbf{y}, \mathbf{X}, \mathcal{H}_i) = \arg\max_{\theta \in \Theta} P(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i) \tag{2.11}$$

Principem tohoto přístupu je, že na základě napozorovaných dat hledáme hyperparametry pro náš model, se kterými on popisuje naše data co nejlépe pro dané modely. Tato metoda ale má řádu nyvýhod. Náš odhad totiž skončí v lokálním maximu a nemusí vždy dosáhnout optimálních hodnot. To samozřejmě vynucuje nás zkoušet různé startovní body, zkoumat citlivost odhadu a patrně ladit parametry hledání. To už nemluvě o ne úplně jednoduchých výpočtech, které také vyžadují jisté množství výkonu a času.

Ze statistiky víme, že jedním ze způsobu, jak najít ML odhad, je použit logaritmickou transformaci a pak najít maximum této funkce. Ukážeme to na konkrétním příkladě a pro to použijeme již definovaný v kapitole (reference) Gaussovský proces s RBF kernelem. Zkusme tedy rozepsat výraz $\log P(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta})$ (člen $\mathcal{H}_i$ můžeme vynechat pro lepší čitelnost). Vzpomeňme si, že při definici Gaussovského procesu jsme použili vícerozměrnou Gaussovskou hustotu 2.6. Zlogaritmujeme tento výraz a obdržíme:

$$\log p(\mathbf{x}) = -\frac{1}{2}\mathbf{y}^\top \Sigma^{-1}\mathbf{y} - \frac{1}{2}\log|\Sigma| - \frac{n}{2}\log 2\pi \tag{2.12}$$

Jediné, co nám tedy zbývá v případě Gaussovského procesu, dosadit za kovarianční matici kernelovou funkci, čímž obdržíme (kernelovou funkci nebo matici???):

$$\log P(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i) = -\frac{1}{2}\mathbf{y}^\top K_y^{-1}\mathbf{y} - \frac{1}{2}\log|K_y| - \frac{n}{2}\log 2\pi \tag{2.13}$$

Chceme-li najít extrém této funkce, musíme tento výraz derivovat podle hyperparametrů, čímž dostáváme:

$$\begin{aligned}\frac{\partial}{\partial\theta_j}\log p(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta}) &= \frac{1}{2}\mathbf{y}^\top K^{-1}\frac{\partial K}{\partial\theta_j}K^{-1}\mathbf{y} - \frac{1}{2}\operatorname{tr}\left(K^{-1}\frac{\partial K}{\partial\theta_j}\right) \\ &= \frac{1}{2}\operatorname{tr}\left(\left(\boldsymbol{K^{-1}}\mathbf{y}\right)\left(\boldsymbol{K^{-1}}\mathbf{y}\right)^\top - K^{-1}\right)\frac{\partial K}{\partial\theta_j}\Bigg),\end{aligned} \tag{2.14}$$

zde je dokonce vidět, proč tento model není optimální z výpočetního hlediska. Je totiž známo, že najít inverzní matici je dost náročná operace. Pro standardní algoritmy platí, že jejich složitost je $O(n^3)$, kde n je velikost matice. V praxi je ovšem n často nabývá dost velkých hodnot. Avšak lze vyhnout výpočtu přímé inverze, místo toho se může použit např. Choleského rozklad, který tetno výpočet může usnadnit. Máme-li spočítanou inverzi, pak je složitost zbylých výpočtů je $O(n^2)$. Další nevýhodou je to, že nemusíme spadnout do globálního optima a skončit v lokálním optimu, což opět vyžaduje větší pozornost.

Type II ML obrazky a rozepsat členy toho logaritmu.

2.4.2 Markov Chain Monte Carlo

Další metodou je Markov Chain Monte Carlo (MCMC). Ta je založena na teorii Markovských procesů. Její princip spočívá v tom, že se snažíme náhodným vzorkováním z rozdělení aproximovat integrál, který neumíme vyřešit analyticky. konkrétně už jsme si říkali, že integrály 2.4 a 2.5 nemusíme umět spočítat kvůli jejich příliš velké dimenze nebo kvůli tomu, že tyto hustoty nemusí být vlastní. A tak místo přímého výpočtu budeme postupně generovat vzorky z těchto rozdělení a postupně zjistíme alespoň hrubou představu o skutečně podobě těchto rozdělení, dokonce můžeme i aproximovat jakékoliv statistiky tohoto rozdělení. Dalším podstatným bodem je to, že MCMC je založeno na tzv. Markovových řetězcích což je posloupnost náhodných veličin, pro kterou platí $p(\theta_{t+1}|\theta_t, \theta_{t-1}, \ldots, \theta_0) = p(\theta_{t+1}|\theta_t)$, kde θ_i pro $i \in \{1, \ldots, T\}$ značí vzorky generované z MCMC. Klíčovým požadavkem je, aby řetězec:

1. byl ergodický, tj. ireducibilní a aperiodický,
2. měl stacionární rozdělení $\pi(\theta)$.

Pak vzorky konvergují k $\pi(\theta)$ a můžeme použit pro odhady:

$$\lim_{T\to\infty} \frac{1}{T} \sum_{t=1}^{T} f(\theta_t) = \mathbb{E}_\pi[f(\theta)],$$

kde $f(.)$ je v tomto případě skutečná hustota. Dnes již existuje cela řada různých variant MCMC algoritmů, jsou to např. *Metropolis-Hastings algoritmus*, *Gibbs sampling* nebo modernější *Hamiltonian Monte Carlo*. Avšak není účelem této práce rozebírat všechny tyto metody dopodrobna. Nicméně o jedné metody toho řekneme trochu víc, a to je varianta *No U-Turn Sampler* využívající princip Hamiltonian Monte Carlo. Tato varianta totiž je velmi užívána a relativně účinná při hledání aposteriorních distribucí. Hamiltonian Monte Carlo řeší problém tím, že využívá gradient aposteriorního rozdělení. Základní myšlenkou je pohyb v parametrickém prostoru.

Tento proces vnímáme jako fyzikální systém:

- θ = poloha částice,
- zavádíme pomocnou momentum proměnnou r (někdy p),
- pohyb řídí Hamiltonovy rovnice s energií:

$$H(\theta, r) = -\log p(\theta|\mathcal{D}) + \frac{1}{2} r^T M^{-1} r,$$

  kde první člen je potenciální energie (negativní log-posterior) a druhý je kinetická energie.

Rovněž si musíme uvědomit, že částice má tendenci se pohybovat po hladinách konstantní energie, a proto zkoumá prostor efektivněji než náhodné procházky, které používají předchozí algoritmy!

Hamiltonian Monte Carlo avšak má dva hyperparametry, které je těžké nastavit:

Algorithm 1 Hamiltonian Monte Carlo

1: Vzorkuj náhodný momentum: $r \sim \mathcal{N}(0, M)$
2: Simuluj Hamiltonovy rovnice pomocí leapfrog integrace na L kroků délky ϵ
3: Metropolis accept/reject krok (zachovává přesnou stacionaritu)

- délka kroku ϵ (step size)
- Počet kroků L (trajectory length)

NUTS řeší oba problémy tím, že automaticky volí délky trajektorie. To znamená, že pokračuje v simulaci, dokud trajektorie nezačne dělat tzv. *U-turn*, což znamená, že částice se po trajektorii otočí a začne se vracet zpět směrem k místu, odkud vyšla. Toto U-turn kritérium lze vyjádřit takto: směr pohybu začne být opačný než směr od startovní pozice, matematicky to znamená $(\boldsymbol{\theta}_{\text{forward}} - \boldsymbol{\theta}_{\text{backward}}) \cdot \mathbf{r}_{\text{forward}} < 0$, kde skalární součin vektoru mezi krajními body trajektorie a momentem je záporný, což signalizuje, že se pohybujeme zpět. A tento princip má dobrý smysl, jakmile se částice vrací zpět, už není efektivní pokračovat, znamená to, že jsme plně prozkoumali danou "energetickou hladinu". Další kroky by jen ztrácely výpočetní čas tím, že by procházely oblasti, které už jsme navštívili. NUTS tedy automaticky zastaví simulaci v optimálním bodě, když jsme prozkoumali maximum užitečného prostoru, ale ještě před tím než začneme ztrácet efektivitu návratem zpět.

Kapitola 3

Prior–Data Fitted Networks

V této kapitole uvedeme samotnou architekturu Prior–Data Fitted Networks (dále jenom PFN). Začneme krátkou motivací, kde popíšeme hlavní ideu této Depp Learning architektury. Následně ukážeme mechanismy, které se schovávají uvnitř PFN, jinými slovy, jak PFN postupně zpracovává vstupní hodnoty, a to vše na příkladě Gaussovských procesů.

3.1 Úvod do Prior–Data Fitted Networks

Prior–Data Fitted Networks je Deep Learning (dále jenom DL) architektura, kterou v roce 2022 uvedli Samuel Muller a Noah Hollmann v článku (REFERENCE). Hlavní motivací pro vznik tohoto modelu je snaha posunout techniky Bayesovské statistiky na další úroveň spolu se strmým rozvojem v oblasti strojového učení. PFN je založen na proslulé architektuře transformer, která je viníkem dnešního pokroku všemožných Deep Learning modelu, a o niž víme díky velkým jazykovým modelům, a není to náhoda, existuje totiž jisté spojení mezi těmito architekturami. Ačkoli samotnou architekturu PFN a její technické nunace budeme podrobně diskutovat až v následující sekci, zde jenom uvedeme její základní charakteristiky. PFN je tzv. encoder–only architektura (tj. neobsahuje dekóder blok, ale pouze enkóder blok transformeru), na kterou je pak napojena softmax hlava, pro vyhodnocování výsledků ve tvaru pravděpodobností. Na vstup PFN dostává sekvenci dat. V případě velkých jazykových modelu by se jednalo o slovy nebo věty, umístěné v jistém pořadí. Tady je ta myšlenka shodná, ale namísto slov dostane síť neuspořádanou sekvenci dat, které získáme z prior rozdělení. Právě toto je důležitý intuitivní pojetí, na kterém je založená celá ta architektura. Stejně jako po jazykovém modelu, který dostal sekvenci slov, budeme chtít aby odhadl celkový kontext a jako výsledek např. i následující neznáme slovo, které by mohlo následovat , budeme i po PFN chtít, aby po předložení sekvenci dat, zkusil uchopit celkový trend (neboli kontext) a předpovědět, co by mohlo následovat dále. Což je velmi relevantní, když zpracováváme časové řady a chceme řešit regresní problém, jak tomu je např. během práce s Gaussovskými procesy.

Celý princip fungování teto metodiky lze popsat následujícím způsobem. Jak již jsme diskutovali dříve, účelem Bayesovského přístupu ve statistice je využití apriorní znalosti nebo informace o jistém jevu v přírodě. A tento princip se dobře prolíná s obecnou úlohou aproximace funkcí pomocí neuronových sítí, protože při jejich návrhu vždy musíme enkódovat aprirorní znalosti o této funkce. To se dělá buď přímo skrz architekturu konkrétního modelu nebo za pomocí různých regularizací. Každopádně je to velmi netriviální úloha, kterou je třeba vždy řešit než začneme řešit kteroukoliv úlohu pomocí DL nástrojů. Jako jedno z možných řešení se nabízí právě tzv. Bayesovská inference, jejíž hlavním předpokladem je, že se jistý jev nebo proces vyskytuje v reálném světě a disponujeme informacemi o jeho chování. Tento předpoklad nám umožňuje definovat následující postup. Uvažujme jev nebo celý proces (to jest prior, pro jednoduchost si pod tím můžeme představit obyčejnou množinu dat), ze kterého mů-

žeme neomezeně vzorkovat simulované datové sady $\mathcal{D}$ reprezentující různé prediktivní úlohy (body). Bez újmy na obecnosti předpokládejme, že tyto datové sady obsahují vstupní příznaky $\mathbf{x} \in \mathbb{R}^d$ a k nim příslušné výstupní třídy $y \in \mathcal{Y}$. Během trénování sítě PFN náhodně vygenerujeme dávku (batch) obsahující K takových datových sad. Pro každou i-tou sadu v dávce ($i \in \{1, \ldots, K\}$) provedeme následující kroky:

1. Zvolíme náhodně velikost kontextu N_i.
2. Vygenerovanou datovou sadu rozdělíme na kontextovou (trénovací) část $\mathcal{D}^{(i)}_{train} = (\mathbf{x}^{(i)}_j, y^{(i)}_j)_{j=1}^{N_i}$ a testovací část obsahující (pro jednoduchost) jeden testovací vzorek $(\mathbf{x}^{(i)}_{test}, y^{(i)}_{test})$. Tedy předložíme modelu data ve tvaru $D^{(i)} = D^{(i)}_{train} \cup \{\mathbf{x}^{(i)}_{test}, y^{(i)}_{test}\}, \forall i \in K$.

Následně natrénujeme náš model q_θ tak, že mu na vstup předložíme kontextovou sadu společně s testovacím příznakem a učíme ho predikovat správný výstup. K tomu minimalizujeme ztrátovou funkci Negative Log-Likelihood (NLL) přes celou dávku K vzorků ve tvaru:

$$L(\theta) = -\sum_{i=1}^{K} \log q_\theta(y^{(i)}_{test} | \mathbf{x}^{(i)}_{test}, \mathcal{D}^{(i)}_{train})$$

Optimalizací této funkce (např. pomocí gradientního sestupu) hledáme parametry $\hat{\theta} = \arg\min_\theta L(\theta)$, čímž se síť učí provádět inferenci in-context na základě poskytnutých dat.

Toto schéma nám umožňuje aproximovat aposteriorní prediktivní rozdělení (PPD) pro libovolný problém, pro který jsme schopni definovat apriorní rozdělení (prior) a vzorkovat z něj data. Zde je namístě si ujasnit terminologii. **Aposteriorní rozdělení (Posterior)** popisuje naši nejistotu ohledně samotného generativního modelu p (nebo jeho parametrů) poté, co jsme pozorovali trénovací data $\mathcal{D}_n$. Matematicky se jedná o podmíněné rozdělení modelu vzhledem k těmto datům, které se značí jako $\Pi(p|\mathcal{D}_n)$. **Aposteriorní prediktivní rozdělení (PPD - Posterior Predictive Distribution)** naproti tomu popisuje nejistotu ohledně nových dat, které model dosud neviděl, jako je například predikce testovacího štítku y pro nový vstupní příznak x, na základě již pozorovaných dat $\mathcal{D}_n$. Vypočítá se jako průměr všech možných podmíněných rozdělení $p(y|x)$ vážených jejich aposteriorní pravděpodobností. Matematicky je definováno pomocí integrálu:

$$\pi(y|\mathbf{x}, \mathcal{D}_n) = \int p(y|\mathbf{x}) d\Pi(p|\mathcal{D}_n).$$

Takže účelem PFN je aproximovat právě PPD.

V praxi to znamená, že model je trénován na široké rodině úloh vygenerovaných z tohoto prioru. Tím se PFN naučí implicitně provádět bayesovskou inferenci nad daným prostorem funkcí.

V případě Gaussovských procesů to vypadá následovně: Jelikož známe přesný tvar rozdělení (např. Gaussovský proces s RBF kernelem), dokážeme z něj nekonečněkrát vzorkovat nová data. Navíc můžeme hyperparametrům RBF kernelu přiřadit vlastní rozdělení (tzv. hierarchický prior), čímž úlohu pro PFN zesložitíme. V této chvíli se model musí naučit generalizovat (tj. najít obecné řešení) přes celou rodinu funkcí pro všemožné hodnoty hyperparametrů. Potom stačí takto natrénovanému modelu předložit několik známých bodů (tzv. kontext) a on nám v jediném průchodu sítí vrátí rozdělení, které chceme predikovat a snaží se co nejvíce aproximovat původní funkci.

3.2 Vnitřní architektura PFN

V této sekci se pokusíme popsat procesy, které se dějí uvnitř PFN transformeru. Naším účelem není poskytnout kompletní popis architektury transformer obecně, za co odpovídají jednotlivé komponenty

nebo parametry. Předpokládáme, že čtenář je již seznámen se základy architektury transformer. Pokud tak tomu není, doporučujeme nastudovat alespoň základní pojmy a principy této architekury. Potřebné zájemce jenom odkážeme na příslušnou kapitolu v knize (REFERENCE UDNERSTANDING DEEP LEARNING). Přesto pevně věříme, že našemu výkladu budou rozumět i laici v oblasti strojového učení, neboť zde nepotřebujeme pokročilé znalosti nebo složité pojmy a zkusíme vše popsat co nejjednodušeji. Pokud se takové ve výkladu objeví, slouží jenom pro odborníci v této oblasti a na porozumění hlavních principu nebudou mít velký vliv. Dále opět podotýkáme, že učelem práce není rozebírat konkretní implementaci PFN a související se s tím nuance. Navíc PFN je relativně mladá architektura, které se neustále vylepšuje a aktualizuje, proto pro nás nemá smysl jít do takových detailů. Aktuální verzi zdrojového kód PFN naleznete zde (REFERENCEpfngit).

V předchozí sekci jsem již zmínili, že PFN je transformer, který se trénuje na obrovském množství synetických datasetů, které vzorkujeme z prioru. Tímto způsobem se PFN naučí provádět Bayesovskou inferenci a po natrénování stačí jeden forward pass sítí a model vrátí kompletní PPD pro nové testovací body na vstupu. V této sekci budeme uvažovat, že naším priorem je Gaussovsky proces s RBF kenrlem. Dále chceme upozornit, že celý proces, který se odehrává uvnitř PFN od vstupu po výstup, popíšeme s použitím konkrétní konfigurace, kterou jsme běžně používali během zkoumání PFN:

1. Dimenze embidingu *EMSIZE = 512*.
2. Počet attention hlav v každé vrstvě *NHEAD = 8*.
3. Dimenze vnitřních skrytých vrstev dopředných neuronových sítí *NHID = 1024*.
4. Počet vrstev *NLAYERS = 6*.

Jelikož se vše chystáme navíc ukázat pro případ $1D$ GP regrese, použijeme úplně klasické nastavení PFN, které uvádějí Muller a Hollmann v (REFERENCEmuller), tj. `features_per_group = 1` a `attention_between_features = False`. První parametr je nastaven na jedničku, protože řešíme $1D$ regresi. Druhý parametr slouží pro zpracování tabulkových dat, pokud je tento parametr zapnutý, jedná se o mírně odlišnou architekturu **TabPFN**, kterou avšak v této práci neuvažujeme, více informací o této architektuře naleznete zde (REFERENCE**TabPFN**). Nakonec uvažujme, že provádíme inferenci pro GP regresi s 20 trénovacími a 50 testovacími body. Teď už máme nachystané formální parametry a nastavení, což nám umožňuje se pustit na samotný popis. Celý proces si rozdělíme na několik fází.

3.2.1 Fáze 1 - Meta–Learning

Pro obecnost ukážeme, jak probíhá učení pro případ, kdy hyperparametry pro RBF kernel nejsou známe a pochází z nějakého rozdělení. Případ fixních hyperparametrů je pouze speciálním případem tohoto problému a všechno, co napíšeme dále, platí i pro tento speciální případ. Meta–Learning je paradigma strojového učení, které spočívá v tom, že namísto toho, abychom model naučili řešit jeden konkrétní fixní problém, naučíme ho řešit celou třídu podobných úkolů. Konkrétně v našem případě by to znamenalo, že model naučíme na základě dat z GP s RBF kernelem, kde mu ukážeme celé spektrum všemožných hyperparametrů, které se v RBF kernelu vyskytují. Potom stačí jenom předložit modelu těch 20 kontext bodů a hned bude schopen aproximovat GP, ze kterého tyto body pocházejí. V předchozí sekci jsem již drobně popsali, jak funguje trénink PFN, formálněji a mnohem podrobněji lze tento proces zapsat takto:

Algorithm 2 Meta-trénink sítě PFN na syntetických GP datech

Require: Model q_θ, apriorní rozdělení hyperparametrů Π, velikost kroku η, velikost dávky m

while není dosaženo konvergence **do**
  for $j = 1, \ldots, m$ **do**
    Vzorkuj hyperparametry GP: $(\ell_j, \sigma_j, \sigma_{n,j}) \sim \Pi$
    Vzorkuj vstupní body $x_1, \ldots, x_N$ rovnoměrně z $[0, 1]$
    Vzorkuj výstupy z GP s daným kernelem: $(y_1, \ldots, y_N) \sim \mathcal{GP}(0,\ k_{\ell_j,\sigma_j}) + \mathcal{N}(0, \sigma^2_{n,j} I)$
    Rozděl na kontextovou sadu $\mathcal{D}_j = \{(x_i, y_i)\}_{i=1}^n$ a testovací body $\{(x_i^*, y_i^*)\}_{i=1}^t$
  end for
  Předej q_θ kontexty $\mathcal{D}_j$ a testovací vstupy x_i^*; model vrátí prediktivní distribuce přes biny
  Spočítej ztrátu (NLL):

$$\mathcal{L}(\theta) = -\frac{1}{m} \sum_{j=1}^{m} \sum_{i=1}^{t} \log q_\theta(y_i^* \mid x_i^*, \mathcal{D}_j)$$

  Aktualizuj: $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}(\theta)$
end while
return $\hat{\theta}$

Tímto způsobem se dostáváme do situace, kdy model vidí miliony různých datasetů generovných z GP prioru a učí se pravidla, dle kterých pak musí uhádnout, jak vypadá posteriorní predikce na základě předložených dat. Tohle je velmi důležitá myšlenka, PFN se neučí aproximovat konkrétní funkce, učí se inferenční pravidlo.

3.2.2 Fáze 2 - Vstup do modelu

Během inference model dostane tři tenzory:

1. Souřadnice trénovacích bodů `x_train`.
2. Naměřené hodnoty na trénovacích bodech `y_train`.
3. Souřadnice testovacích bodů `x_test`.

Jak už plyne z logiky věci, `x_train` a `y_train` jsou dva různé objekty, první říká, kde jsme měřili (souřadnice na ose x), a druhý říká, co jsme v těchto místech naměřili. Tak dohromady tvoří datové body (x_i, y_i). Souřadnice `x_test` nám říká, kde chceme ty predikce, přitom model by měl vrátit pravděpodobnostní rozdělení pro příslušné y–hodnoty, ale k tomu se dostaneme později v naším výkladu. Následně se `x_train` a `x_test` spojí do jedné sekvenci a model s nimi pracuje jako s jedním objektem. Přičemž modelu současně explicitně řekneme, že prvním 20 souřadnic jsou trénovací hodnoty a zbytek jsou testovací (50 testovacích souřadnic), jak jsme si to definovali na začátku sekce. Pro y–hodnoty je ta situace podobná, vytvoříme jednou sekvenci pro `y_train` a `y_test` hodnoty, až na to, že místo test hodnot vložíme `NaN` hodnoty, protože právě ty musí model odhadnout a nechceme je do modelu vkládat v žádné podobě. Pro zájemce uvedeme, jak to pak vypadá v kódu. V metodě `forward()` se `x_train` a `x_test` spojí pomocí funkce konkatenace:

```
x = torch.cat((x_train, x_test), dim=1)  # (1, 70, 1)
```

Jedná se o tenzor, kde první dimenze je počet datasetů, které zpracováváme naraz, druhá dimenze je počet bodů v sekvenci a třetí dimenze je počet příznaků, který jsme na začátku této sekce nastavili na 1. Pro y–hodnoty analogicky:

```
y = torch.cat((y_train, NaN * torch.zeros(1, 50, 1)), dim=1)  # (1, 70, 1)
```

Zde je explicitně vidět, že test hodnoty se nahradí za `NaN`. Dále si myslíme, že je namístě říct jedno upřesnění, aby došlo k nedorozumění. To, že modelu v y–hodnotách předkládáme `NaN` neznamená, že pak budeme chtít tyto hodnoty nahradit za odhady někde v průběhu zpracovávání, je to čistě technická záležitost. Zůstanou `NaN` po celou dobu. Výsledky obdržíme v jiné podobě, opět, budeme o tom diskutovat dále.

Dále následuje tzv. *Feature grouping*, o parametru, který určuje počet příznaku jsme také zmiňovali na začátku sekce. Připomínáme, že pro náš jednoduchý případ 1D regrese je nataven na 1. Tento mechanismus slouží hlavně pro případ, kdy chceme zpracovávat tabulkové hodnoty, anebo pracovat s vícerozměrnými daty, ale to se týká hlavně verze modelu **TabPFN**. V tabulkových datech s 12 sloupci (věk, plat, výška, ...) model potřebuje tyto sloupce nějak zpracovat. Parametr `features_per_group` určuje, kolik sloupců se seskupí do jednoho tokenů. Pro náš 1D GP je grouping triviální: máme 1 přiznak, tedy 1 grupu. Tenzor x se tak přeformátuje z $(1, 70, 1)$ na $(1, 70, 1, 1)$.

Dalším krokem je převod dat na vstupu na tzv. *embeding*, tj. do řeči, které rozumí transformer. Ve skutečnosti se jedná o vytváření nového prostoru, ve kterém pak PFN bude operovat s daty, a kterému rozumí víc, než kdyby musel pracovat pouze se sekvenci dat.

X–encoder:

Zde to je jednoduchá lineární vrstva, která vezme jednu konkretní číselnou hodnotu x–souřadnice a vynásobí ji maticí vah 1×512 (parametr velikosti embedingu jsme nastavili nahoře). Výsledkem je 512–dimenzionální vektor (*embeding*). Tímto způsobem jsme zakódovali polohu bodu v tom novém prostoru, ze kterého model umí jednoduše číst a operovat v něm. Výstupem je `embedded_x` tvaru $(1, 70, 1, 512)$

Y–encoder:

Pro y–hodnoty je ta situace zajímavější, jelikož v předchozím kroku, jsme si řekli, že pro testovací hodnoty modelu předložíme `NaN` hodnoty. A teď otázkou je, jak tyto hodnoty zakódovat do *embedingu*. K tomu slouží metoda `NanHandlingEncoderStep`, která nejdříve transformuje skalární y–hodnotu na vektor délky 2:

1. Pro trénovací pozice (y existuje): $[y, -1]$ — skutečná hodnota a indikátor "data jsou k dispozici".
2. Pro testovací pozice (y = `NaN`): $[0, +1]$ — nulová hodnota a indikátor "data chybí".

Pak lineární vrstva opět převede dvourozměrný vektor do 512–dimenzionálního prostoru. Výstupem je `embedded_y` tvaru $(1, 70, 512)$, tady je o jeden parametr méně, protože příznaky mezi skupinami se nevztahují na y–hodnoty.

V této chvíli máme dva embedingy, cílem ale je, mít jeden hromadný embdeding, který v sobě nese informace o souřadnici x a odpovídající ji hodnotě y. Zde opět existují dva způsoby implementace v PFN: pro `attention_between_features=False` a `attention_between_features=True`, kde poslední verze je opět určena pro práci s vícedimenzionálními daty. Pro tu první verzi je postup velmi jednoduchý – `embedded_x` a `embedded_y` jednoduše sečteme. Výsledkem je jeden token pro odpovídající jednu konkretní pozici, tj, tento token kombinuje informaci o poloze x a hodnotě y v jednom vektoru. Sčítání tady funguje, protože oba embidingy jsou již stejného tvaru a jsou ve stejném 512–dimenzionálního prostoru a model bude umět z tohoto součtu dekódovat obě informační položky díky předchozímu postupu. V kódu to pak formálně vypadá následovně:

```
embedded_input = embedded_x + embedded_y.unsqueeze(2)
# (1, 70, 1, 512) + (1, 70, 1, 512) -> (1, 70, 1, 512)
```

Druhý režim zde již popisovat nebudeme, zájemce odkážeme na zdrojové kódy na GitHub, které jsme uvedli na začátku kapitoly.

3.2.3 Fáze 3 – Transformer vrstvy

Tenzor `embedded_input` pak musí projít 6 transformer vrstvami (jejich počet jsme též definovali na začátku sekce). Jakmile embedingy vstupují do transformer vrstev je zvykem jim říkat token. To znamená, že objektům `embedded_x` a `embedded_y` bychom říkali `x_token` a `y_token`. Ačkoliv jsme tyto dva embedingy sloučili do jednoho, pořad musíme na ně nahlížet jako na samostatné objekty, protože naším cílem je nakonec získat `y_token`, který v sobě bude obsahovat predikce pro testovací hodnoty. Sloučení se tedy provádí pouze pro technické potřeby architektury. Vraťme se ale zpátky k věci. Každá vrstva v naší konfiguraci provádí dvě hlavní operace:

1. Self–attention přes datové body (neplést s attention between features)
2. Feedforward MLP

Self–attention:

Tenzor se transponuje na `(1, 1, 70, 512)`, aby sekvence 70 bodů byla na dimenzi `dim=2` (standardní konvence pro attention). Multi-head attention s 8 hlavami pak počítá:

Pro každou hlavu h (z 8):

$$Q^{(h)} = x \cdot W_Q^{(h)}, \quad K^{(h)} = x \cdot W_K^{(h)}, \quad V^{(h)} = x \cdot W_V^{(h)}$$

kde $W_Q, W_K \in \mathbb{R}^{512\times 64}$ a $W_V \in \mathbb{R}^{512\times 64}$ (protože $512/8 = 64$).

Klíčový detail: K a V se počítají pouze z trénovacích bodů. V kódu:

```
attention_src_x = x[:, :single_eval_pos].transpose(1, 2)
# first 20 positions
```

Attention matice je pak:

$$A = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{64}}\right)$$

Pro trénovací body (pozice 0–19) je Q i K ze stejných 20 bodů, takže matice je 20×20 — trénovací body se dívají na sebe navzájem.

Pro testovací body (pozice 20–69) Q pochází z testovacích bodů, ale K pochází z trénovacích bodů. Matice je 50×20 — testovací body se dívají na trénovací body.

Tohle je přímá analogie ke GP predikci. Ve formuli $\mu_* = k(x_*, X)K^{-1}y$ se testovací body taky „dívají“ na trénovací body přes kernel $k(x_*, X)$. Rozdíl je, že GP používá explicitní RBF kernel, zatímco PFN se naučil vlastní implicitní „kernel“ zakódovaný ve váhách W_Q a W_K.

Výstup attention:

$$\text{Output}^{(h)} = A \cdot V^{(h)}$$

Všech 8 výstupů (každý $\in \mathbb{R}^{64}$) se zřetězí a projde výstupní projekcí W_O:

$$\text{MultiHead} = \text{Concat}(\text{Output}^{(1)}, \ldots, \text{Output}^{(8)}) \cdot W_O$$

Po attention následuje reziduální spojení (vstup + výstup attention) a LayerNorm — normalizace, která stabilizuje trénink. Reziduální spojení je technika, která přidává původní vstupní reprezentaci (třeba embedding datového bodu) k výstupu nelineární vrstvy (například MLP) těsně předtím, než tento výsledek pošle dál do sítě. Reziduální spojení zajišťuje, že model nezačíná optimalizaci chaotickým hádáním,

nýbrž bezpečným průtokem dat, ke kterému postupně přidává potřebnou komplexitu. Co se tyče LayerNorm, ten normalizuje embedding na nulový průměr a jednotkový rozptyl přes posledních d dimenzí (v našem případě 512). Pro vektor x délky 512:

$$\text{LayerNorm}(x) = \frac{x - \mu}{\sigma + \epsilon},$$

kde μ a σ jsou průměr a směrodatná odchylka přes 512 komponent vektoru a $\epsilon = 10^{-5}$ je malá konstanta pro numerickou stabilitu.

MLP – Multi-Layer Perceptron:

MLP je dvouvrstvá dopředná síť:

$$\text{vstup (512)} \;\rightarrow\; \text{Linear}(512 \rightarrow 1024) \;\rightarrow\; \text{GELU} \;\rightarrow\; \text{Linear}(1024 \rightarrow 512) \;\rightarrow\; \text{výstup (512)}$$

Vezme 512-dimenzionální vektor, rozšíří ho do 1024 dimenzí přes lineární vrstvu, aplikuje nelineární aktivaci GELU (hladká varianta ReLU), a zase zúží zpět na 512.

GELU (Gaussian Error Linear Unit) je funkce

$$\text{GELU}(x) = x \cdot \Phi(x),$$

kde Φ je distribuční funkce standardního normálního rozdělení. Pro velká kladná x se chová jako identita, pro velká záporná x vrací přibližně nulu.

Proč nakonec musíme přidávat MLP je pro zkušeného ve strojovém učení čtenáře poněkud triviální otázka, vysvětlíme i pro ostatní čtenáře. Attention je totiž lineární operace (ačkoliv se to nezdá), počítá pouze vážené průměry. MLP do tohoto procesu přidává nelinearitu, bez něj by transformer mohl dělat jenom lineární transformace s embedingy. Avšak ve strojovém učení je to nedostačující, chceme zachytit a složitější transformace a operace, čímž zvyšujeme generalizační schopnosti modelu. A právě díky této vlastnosti může PFN aproximovat např. inverzi matice $\mathbf{K}^{-1}$ v GP. Váhy druhé lineární vrstvy jsou inicializovány na nulu. To znamená, že na začátku tréninku MLP vrací nulový vektor, a tak se tato vrstva na začátku chová jako identita. To se však mění v průběhu tréninku. Model začíná od jednoduchého stavu a postupně se učí komplexnější transformace. Po MLP opět následuje reziduální spojení a normalizace.

Toto se opakuje tolik, kolik vrstev obsahuje model, v našem případě 6 krát. Po každé vrstvě se embedding každého bodu obohacuje o informaci z ostatních bodů (přes attention) a transformuje (přes MLP). Po 6 vrstvách embedding testovacího bodu obsahuje dostatečnou informaci k tomu, aby dekóder z něj vyrobil prediktivní distribuci.

Po průchodu 6 vrstvami dostáváme opět nový tenzor. Ten obsahuje zpracované reprezentace jak trénovacích, tak i testovacích bodů. Naším úkolem teď by bylo extrahovat testovací hodnoty y (neboli y–tokeny, jak jsme diskutovali výše), respektive jejich predikce, které model postupně vyrobil. Zbytek je pro účely inference nepodstatný. Platí totiž, že celá architektura PFN je postavena tak, že se právě v y-tokenu postupně akumulují informace přes všechny vrstvy transformeru přes attention z x–tokenů a z kontextu trénovacích bodů. Na konci celého procesu y–token pro testovací pozici obsahuje 512–dimenzionální vektor, který implicitně kóduje celou posteriorní prediktivní distribuci (PPD) pro daný trénovací bod. Uveďme opět pro zájemce, jak tento proces vypadá uvnitř, což by mělo pomocí k pochopení, co se v tomto kroku odehrálo:

```
# Split into train and test part
test_encoder_out = encoder_out[:, current_context_len:, :]
# (1, 50, 1, 512) -- test positions (20-69)

train_encoder_out = encoder_out[:, :current_context_len, :]
# (1, 20, 1, 512) -- train positions (0-19)
```

Proměnná `current_context_len` je rovna `single_eval_pos = 20`.

V dalším kroku se extrahuje y-token — poslední token v dimenzi feature groups:

```
test_encoder_out = test_encoder_out[:, :, -1, :]
# (1, 50, 512) -- only y-token for test positions
```

3.2.4 Fáze 4 – Dekóder, převod embdedingu na logity.

Jsme si již řekli, že PFN je encoder–only architektura, tj. decoder blok, který je součastí obecné architektury transformer, v něm chybí. Decoder blok ale je napojen na hlavu, která v sobě obsahuje opět MLP vrstvu a softmax vrstvu. Tyto součástky ma ovšem potřebovat budeme. Na konci předchozí fáze jsme získali tenzor `test_encoder_out` o tvaru $(1, 50, 512)$, který v sobě obsahuje pro každý z 50 testovacích bodů jeden 512–dimenzionální embeding. Teď potřebujeme z tohoto vektoru vytvořit prediktivní distribuci. To se opět udělá pomocí MLP (jednoduchá dopředná síť), která je doplněná o jednu novinku. Celkově schéma vypadá následovně:

vstup (512) $\rightarrow$ Linear(512 $\rightarrow$ 1024) $\rightarrow$ GELU $\rightarrow$ Linear(1024 $\rightarrow$ `num_bars`) $\rightarrow$ výstup (`num_bars`)

Jednotlivé prvky v schématu, který jsme uvedli na začátku sekce, plní následující funkci:

1. `Linear(512 -> 1024)`: Vezme 512-dimenzionální embedding a promítne ho do 1024 dimenzí. To je lineární transformace — násobení maticí vah $W_1 \in \mathbb{R}^{512\times1024}$.

2. `GELU()`: Aplikuje nelineární aktivaci. Bez ní by dvě lineární vrstvy za sebou byly ekvivalentní jedné lineární vrstvě (součin matic je matice). GELU umožňuje decoderu zachytit nelineární vztahy mezi embeddingem a výstupní distribucí.

3. `Linear(1024 -> num_bars)`: Zúží z 1024 dimenzí na `num_bars` (řekněme 1000). Výstupem je 1000 čísel — **logity** — jedno pro každý bucket.

Je vidět, že tady se objevuje nová proměnná `num_bars`, jedná se totiž o analogii binu v histogramech, který se zavádí pro tzv. **BarDistribution**.

BarDistribution

BarDistribution je po–částech konstantní pravděpodobnostní rozdělení. V článku (REFERENCE-muller) se mu říká *Riemann distribution*, jedná se o diskretizaci spojité distribuce. Vizuálně to vypadá takto: Avšak je důležité odlišit tento koncept od klasického histogramu. Histogram odhaduje hustotu z dat. Hlavním principem je, že počítáme klik naměřených hodnot padlo do každého binu, a to určuje výšku jednotlivých sloupců. BarDistribution je naopak výstup modelu, tj. my neděláme měření a nepočítáme, kolik hodnot padlo do toho či oného binu. Právě samotný model určuje výšku sloupců dle nějakých svých pravidel. Neuronová síť na výstupu vrátí n čísel, každé číslo říká, jak vysoký má být jeden sloupec. To jest distribuce určuje síť, to umožňuje pak modelovat pestrou škálu různých rozdělení a nejsme omezení jenom normálním, exponenciálním rozdělením apod. Binům se v BarDistribution říká *bucket*. V implementaci je BarDistribution definován pomocí hranic (parametr `borders`), tím definujeme interval, na kterém je tato funkce definována. Může to vypadat například takto:
`borders = [-3.0, -2.994, -2.988, ..., 0.0, ..., 2.988, 2.994, 3.0]` Mezi sousedními hranicemi leží jeden bucket. Bucket č. i pokrývá interval $[b_i, b_{i+1})$ a má šířku $w_i = b_{i+1} - b_i$. V naší konfiguraci jsou borders typicky rovnoměrně rozložené, takže všechny buckety mají stejnou šířku. Implementace ale podporuje i nerovnoměrné dělení (hustší buckety v zajímavých oblastech, řidší na krajích).

Vraťme se zpátky k dekódování. Jednotlivé prvky v schématu, který jsme uvedli na začátku sekce, plní následující funkci:

1. `Linear(512 -> 1024)`: Vezme 512-dimenzionální embedding a promítne ho do 1024 dimenzí. To je lineární transformace — násobení maticí vah $W_1 \in \mathbb{R}^{512\times1024}$.

2. `GELU()`: Aplikuje nelineární aktivaci. Bez ní by dvě lineární vrstvy za sebou byly ekvivalentní jedné lineární vrstvě (součin matic je matice). GELU umožňuje decoderu zachytit nelineární vztahy mezi embeddingem a výstupní distribucí.

3. `Linear(1024 -> num_bars)`: Zúží z 1024 dimenzí na `num_bars` (řekněme 1000). Výstupem je 1000 čísel, říkáme jim **logity**, jeden pro každý bucket.

Logit v tomto případě je jenom surové číslo z poslední lineární vrstvy našeho dekóderu. Může to být jakékoliv reálné číslo. Důležité ale je, že nemají žádné jednotky, nesčítají se na jedničku a ne této etapě nemají žádnou přímou pravděpodobnostní interpretaci. Logit je pouze signál pro určitý bucket, do kterého spadne, jak moc si model myslí, že skutečná hosnota padne do daného intervalu. Čím vyšší je logit, tím víc si model jistý. Ale pro lepší interpretovatelnost musí tyto hodnoty nejdříve projít softmax vrstvou. Uvažujeme–li, že máme 1000 bucketů, softmax převede 1000 logitů na 1000 pravděpodobností:

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^{1000} e^{z_j}}$$

Logicky pak součet všech pravděpodobností je přesně 1 a vyšší logit implikuje vyšší pravděpodobnost. Hustotu pravděpodobnosti potom definujeme jako $f_i = \frac{p_i}{w_i}$, kde w_i je šířka bucketu. Širší bucket musí mít nižší hustotu při stejné pravděpodobnosti, aby distribuce dávala smysl jako aproximace spojitého rozdělení. Při rovnoměrných bucketech je tento rozdíl jen konstantní škálování. Evidentně pak můžeme dopočítat i všechny potřebné diskriptivní charakteristiky jako střední hodnotu, rozptyl atd. Jak jsme již mínili dříve, neklademe na BarDistribution žádná omezení co se tyče tvaru výsledné distribuce. Avšak v kontextu GP inference výstup typicky vypadá přibližně jako gaussovký zvon. To proto, že GP posteriorní prediktivní distribuce je přesně normální rozdělení. Pokud PFN dobře aproximuje GP posterior, buckety budou mít tvar zvonu. Ale model to nikdo nenutí, naučil se to sám, protože loss funkce, se kterou byl model trénován, ho odměňuje za to, že jeho distribuce co nejlépe odpovídá skutečné posteriorní distribuci.

Dále poskytneme krátký komentář, proč během trénování PFN volíme pro loss funkci právě Negative Log–Likelihood (NLL) tvar. NLL je ztrátová funkce, která měří kvalitu predikované distribuce. Postupuje následovně:

1. Model vyprodukuje logity $z_1, \ldots, z_{1000}$.

2. Softmax je převede na pravděpodobnosti $p_1, \ldots, p_{1000}$.

3. Pravděpodobnosti se převedou na hustoty: $f_i = p_i/w_i$.

4. Najde se bucket k, do kterého padne skutečné y_{test}.

5. Loss pro tento bod je: $\mathcal{L} = -\log(f_k) = -\log(p_k) + \log(w_k)$

Člen $\log(w_k)$ je konstanta (borders se nemění), takže gradient teče jen přes $-\log(p_k)$. Minimalizace NLL je ekvivalentní maximalizaci likelihood, to znamená, že model se učí přiřadit co nejvyšší hustotu tam, kde skutečné y leží.

Otázkou je, proč bychom nemohli volit např. tvar Mean Square Error (MSE) jako loss funkci. MSE loss by měřil jen $(y_{\text{pred}} - y_{\text{true}})^2$, tj. chybu bodového odhadu. NLL přes BarDistribution měří kvalitu celé distribuce. Model se učí nejen správný střed, ale i správnou šířku a tvar nejistoty. To je přesně

to, co potřebujeme pro aproximaci GP posterioru, chceme celou PDD, ne pouze průměr. Navíc, jak se ukáže dále (REFERENCEnagler), minimalizace NLL vede k optimálnímu řešení, které je Bayesovská posteriorní prediktivní distribuce, tedy přesně to, co GP analyticky počítá.

Na konci sekce zkusíme shrnout kompletní postup. Na úplném začátku jsme modelu předložili sadu trénovacích a testovacích bodů, přitom ale model nedostal `y_test` a účelem bylo ho naučit tyto hodnoty predikovat. PFN postupně zpracovalo a přetransformerovalo skrz svoje vrstvy `x_train`, `y_train` a `x_test`. To znamena, že model zpracoval mu předložená informace jako celek naraz. Během toho se PFN naučilo pravidlo, jak se chovají různé GP posteriory pro různé parametry. Na konci jsme dostali pravděpodobnostní rozdělení pomocí převodu na logity a použití BarDistribution. Výsledek jsme vždy porovnali se skutečným GP a jako metriku, jak moc dobře PFN odhaduje skutečnost, jsme použili NLL, který bychom chtěli minimalizovat. Po dostatečném tréninku jsme pak dostali model, který na základě předložených dat ze skutečneho GP prioru, které nikdy neviděl, je schopen odhadnout, odkud pochází a jak vypadá funkce, ze které pochází.

3.3 Teoretické statistické vlastnosti PFN

Nagler (2023) ukazuje, že pokud je trénovací loss NLL (negative log-likelihood, viz Fáze 8) a model má dostatečnou kapacitu, optimální řešení konverguje k Bayesovské posteriorní prediktivní distribuci. To znamená, že správně natrénovaný PFN je teoreticky ekvivalentní analytickému GP posterioru.

Závěr

Text závěru....

Literatura

[1] S. Allen, J. W. Cahn: *A microscopic theory for antiphase boundary motion and its application to antiphase domain coarsening*. Acta Metall., 27:1084-1095, 1979.

[2] G. Ballabio et al.: *High Performance Systems User Guide*. High Performance Systems Department, CINECA, Bologna, 2005. `www.cineca.it`

[3] J. Becker, T. Preusser, M. Rumpf: *PDE methods in flow simulation post processing*. Computing and Visualization in Science, 3(3):159-167, 2000.